

Verifica di Tecnologie e progettazione di sistemi informatici e di telecomunicazioni

Nome Cognome: _____ Classe: _____ Data: _____

**Per ogni domanda scegli le opzioni corrette (una o più, a seconda della domanda).
+1 ogni opzione corretta scelta; 0 ogni opzione corretta non scelta; -0.25 ogni opzione errata scelta.**

1. Le Application Programming Interface fanno astrazione perché:

- sono adatte a veicolare dati formattati in linguaggi come WSDL;
- forniscono dati astratti, impossibili da comprendere per un umano;
- forniscono una risposta senza preoccuparsi dei meccanismi di elaborazione;
- nascondono i parametri mandandoli in POST;

2. Le API sono:

- i componenti dell'interfaccia utente;
- mezzi di comunicazione per applicazioni;
- principalmente XML e JSON;
- dei tunnel privati cifrati;

3. Tra i seguenti, NON sono protocolli di rete:

- SMTP;
- HTTP e HTML;
- WSDL;
- DOM;

4. L'uso di WS nello sviluppo di un'applicazione:

- porta a un'esperienza utente notevolmente più lenta;
- impedisce l'uso di altre API;
- richiede sempre accesso alla rete;
- può tornare utile per scrivere meno codice;

5. Le API:

- sono un tipo di Web Services, come DOM;
- usano obbligatoriamente HTTP per il trasferimento di dati;
- possono usare SMTP per il trasferimento di dati (es. in SOAP);
- possono usare SMTP per il trasferimento di dati (es. in Outlook);

6. I Web Services formattano i dati principalmente con:

- XML;
- HTML;

- JSON;
- Java;

7. I Web Services:

- sono interfacce machine-to-machine;
- forniscono risorse in formato machine-readable;
- sono Application Projects for Interaction in contesto web;
- forniscono pagine web dinamiche destinate all'utente finale;

8. SOAP:

- è un protocollo per lo sviluppo di API che non necessitano della rete;
- è un'architettura orientata ai servizi paralleli;
- fu il primo modo di realizzare Web Services;
- implica maggiore overhead;

9. SOAP, era inizialmente acronimo di:

- Services-Oriented Architecture Protocol;
- Services-Oriented Architecture Program;
- Simple Object Access Protocol;
- Solo Object Access Protocol;

10. I SOAP WS:

- sono sempre stateless;
- prevedono "buste" da trasferire mediante WSDL;
- prevedono l'uso di WSDL per la descrizione delle interfacce;
- prevedono "buste" da trasferire mediante HTTP;

11. SOAP ha i seguenti vantaggi:

- permette di pubblicare descrizioni dell'interfaccia;
- dipendenza dalla piattaforma;
- semplicità d'uso, anche in browser;
- overhead maggiore rispetto a REST, perché aggiunge passaggi all'incapsulamento;

12. REST:

- è uno specifico protocollo web basato su TCP;
- prevede l'uso di XML, come SOAP, e impone la scrittura di Web Services con JavaScript;

- è oggi l'unico modo per implementare Web API;
 - prevede lo scambio di dati in formato JSON;
- 13. I RESTful Web Services:**
- prevedono l'invio di HTTP Response usando solo GET, POST, PUT o DELETE;
 - prevedono lo scambio di dati in formati machine-readable;
 - rispettano i 6 pilastri dell'architettura REST;
 - espongono URI non gerarchiche e che non necessitano di estesa documentazione;
- 14. REST è acronimo di REpresentational State Transfer e:**
- suggerisce il trasferimento di pacchetti request/response senza stato;
 - suggerisce design stateless dei servizi;
 - invita a mantenere lo stato lato server;
 - tutte le precedenti sono false;
- 15. Tra i seguenti, sono pilastri di REST:**
- l'uso espanso dei metodi HTTP;
 - lo scambio di dati mediante RPC;
 - l'uso dei metodi POST, GET, PUT e DELETE di HTTP;
 - l'uso di query string nelle URI;
- 16. REST:**
- è il protocollo di base per la realizzazione di SOA;
 - disincentiva l'uso di HTTP, perché insicuro;
 - incentiva l'uso delle query string nelle URI;
 - è più semplice se confrontato con SOAP;
- 17. In ottica RESTful, per creare la risorsa utente con id 123:**
- si deve produrre una HTTP Request con metodo DELETE alla URI `/users/123`;
 - si deve produrre una HTTP Request con metodo POST alla URI `/users/` e con dati opportuni nel payload;
 - si deve produrre una HTTP Response con metodo POST alla URI `/users/list.php?id=123` e con dati opportuni nel payload;
- non si può fare nulla, perché serve per forza ricorrere all'uso delle query string;
- 18. In ottica RESTful, la HTTP Request con metodo GET alla URI `/users/123`:**
- legge la risorsa utente con id 123;
 - non è una richiesta RESTful, poiché l'URI è troppo corta;
 - aggiunge la risorsa utente con id 123 alla lista di utenti;
 - legge le risorse utenti 1, 2 e 3;
- 19. L'architettura client-server (single-server):**
- è estremamente decentralizzata;
 - prevede che sia il server a esporre le interfacce dei Web Services;
 - è un modello iniziale di applicazione distribuita;
 - è un modello che comprende l'architettura peer-to-peer;
- 20. Un'applicazione distribuita sarebbe la soluzione ideale per:**
- la sicurezza in termini di confidenzialità;
 - la semplicità di progettazione;
 - nessuna di quelle sopra;
 - la sicurezza in termini di disponibilità;
- 21. Esempi di applicazioni distribuite sono:**
- Skyscanner;
 - solo le applicazioni che dividono su calcolatori diversi i tre aspetti UI, logica applicativa e dati;
 - Google Drive;
 - le applicazioni che usano API;
- 22. Una SOA:**
- impone l'esistenza di un ESB, ovvero un server di autenticazione degli utilizzatori di servizi;
 - prevede l'esistenza di un ESB, in grado di fare da singolo punto d'accesso alle interfacce dei servizi;
 - usa obbligatoriamente SOAP;
 - è un'architettura che impedisce l'utilizzo di servizi forniti da altre applicazioni;

Punteggio: ____/30
Alcune domande valgono più di altre!